

UNIVERSIDAD MAYOR DE SAN ANDRES
FACULTAD DE CIENCIAS PURAS Y
NATURALES
CARRERA DE INFORMATICA

APELLIDOS Y NOMBRES: GOMEZ JIMENEZ CAROLINA

CI: 8423403

MATERIA: PROGRAMACION WEB 3 – INF 133

FECHA DE ENTREGA: 3 DE JUNIO DEL 2026

```

//Primero se importan express y mysql
const express = require("express");
const mysql = require("mysql2");
const app = express();
app.use(express.json());
// Conectando a MySQL
const db = mysql.createConnection({
  host: "localhost",
  user: "root",
  password: "",
  database: "tienda"
});
db.connect((err) => {
  if (err) {
    console.error("Error de conexión:", err);
    return;
  }
  console.log("Conectado a MySQL");
});
// 1. Crea un endpoint POST /categorias que permita registrar una nueva categoría enviando
// nombre y descripción en el body de la petición.
app.post("/categorias", (req, res) => {
  const { nombre, descripción } = req.body;
  const sql =
    "INSERT INTO categorias(nombre, descripción) VALUES (?, ?)";
  db.query(sql, [nombre, descripción], (err, result) => {

```

```

    if (err) {
        return res.status(500).json(err);
    }
    res.status(201).json({
        mensaje: "Categoría creada correctamente",
        id: result.insertId
    });
});

});

//2. Crea un endpoint GET /categorias que devuelva todas las categorías
//registradas en la base de datos.
app.get("/categorias", (req, res) => {
    const sql = "SELECT * FROM categorias";
    db.query(sql, (err, rows) => {
        if (err) {
            return res.status(500).json(err);
        }
        res.json(rows);
    });
});

//3. Crea un endpoint GET /categorias/:id que devuelva la categoría con el ID
//especificado y además, incluya todos los productos que pertenecen a esa
//categoría.
app.get("/categorias/:id", (req, res) => {
    const { id } = req.params;
    const sql = `

```

```
SELECT
```

```
  c.id,
```

```
  c.nombre AS categoria,
```

```
  c.descripcion,
```

```
  p.id AS productoid,
```

```
  p.nombre AS producto,
```

```
  p.precio
```

```
FROM categorias c
```

```
LEFT JOIN productos p
```

```
  ON c.id = p.categoriaId
```

```
WHERE c.id = ?
```

```
`;
```

```
db.query(sql, [id], (err, rows) => {
```

```
  if (err) {
```

```
    return res.status(500).json(err);
```

```
  }
```

```
  if (rows.length === 0) {
```

```
    return res.status(404).json({
```

```
      mensaje: "Categoría no encontrada"
```

```
    });
```

```
  }
```

```
  res.json(rows);
```

```
});
```

```
});
```

//4. Crea un endpoint PATCH /categorias/:id que permita actualizar todos

//los datos de la categoría con el ID especificado.

```

app.patch("/categorias/:id", (req, res) => {
  const { id } = req.params;
  const { nombre, descripcion } = req.body;
  const sql = `
    UPDATE categorias
    SET nombre = ?,
        descripcion = ?,
        updatedAt = NOW()
    WHERE id = ?
  `;
  db.query(sql, [nombre, descripcion, id], (err, result) => {
    if (err) {
      return res.status(500).json(err);
    }
    if (result.affectedRows === 0) {
      return res.status(404).json({
        mensaje: "Categoría no encontrada"
      });
    }
    res.json({
      mensaje: "Categoría actualizada correctamente"
    });
  });
});

```

//5. Crea un endpoint DELETE /categorias/:id que elimine la categoría indicada

//y, al mismo tiempo, elimine automáticamente todos los productos que

//pertenecen a esa categoría.

```
app.delete("/categorias/:id", (req, res) => {  
  const { id } = req.params;  
  const sql = "DELETE FROM categorias WHERE id = ?";  
  db.query(sql, [id], (err, result) => {  
    if (err) {  
      return res.status(500).json(err);  
    }  
    if (result.affectedRows === 0) {  
      return res.status(404).json({  
        mensaje: "Categoría no encontrada"  
      });  
    }  
    res.json({  
      mensaje: "Categoría eliminada correctamente"  
    });  
  });  
});  
});
```

//el servidor seleccionado

```
app.listen(3000, () => {  
  console.log("Servidor ejecutándose en http://localhost:3000");  
});
```